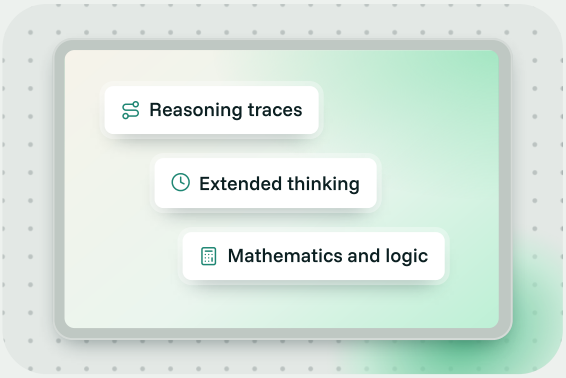
Last updated August 2026

Reasoning Models

How test-time compute lets language models think through hard problems, why chain-of-thought alone was not enough, and what the trade-offs look like.



13 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 8 of 10

What Is a Reasoning Model?

A standard language model generates tokens one at a time, left to right, without the ability to go back and revise. Each token is produced once and becomes part of the context for the next token. This works well for text generation, summarisation, and extraction. It works poorly for problems that require planning, backtracking, or working through multiple steps before committing to an answer.

A reasoning model is a language model trained to produce an extended chain of thought before generating its final answer. Instead of jumping directly from the question to the answer, the model writes out its reasoning process. It breaks the problem into steps, checks each step, considers alternatives, and sometimes corrects itself before arriving at a conclusion. The chain of thought is generated as intermediate tokens that are produced before the final answer.

The core idea behind reasoning models is that giving the model more tokens to think with, meaning more intermediate computation, improves its performance on hard problems. This is called test-time compute or inference-time compute. A standard model spends a fixed amount of compute per token regardless of question difficulty. A reasoning model scales its compute with the difficulty of the problem, spending more tokens on harder questions and fewer on easier ones.

Key principle

Reasoning models trade inference cost for capability. A standard model generates an answer immediately. A reasoning model spends additional tokens thinking through the problem first. The extra tokens are where the reasoning happens, and the model's training teaches it to use those tokens productively rather than just padding.

Why Reasoning Models Exist

Standard language models have a well-documented weakness. They struggle with tasks that require multi-step reasoning, where each step depends on the result of the previous one. Math problems, logic puzzles, coding challenges, and scientific questions all require this kind of sequential thinking. A standard model that generates the answer directly often gets it wrong because it cannot look ahead and check its work.

Chain-of-thought prompting, covered in the prompt engineering guide, was an early attempt to address this. By asking the model to show its work, you could improve its performance on reasoning tasks. The model writes out intermediate steps, and those steps become part of the context that guides the final answer. This works because each token the model generates becomes part of the context for subsequent tokens, so writing out reasoning steps gives the model a richer context to draw from when it produces the answer.

Chain-of-thought prompting had limits. The model was still a standard model trained on next-token prediction. It had not been specifically trained to reason effectively, so its chains of thought were sometimes shallow, circular, or wrong. The model would write out reasoning that looked correct but contained logical errors, and then arrive at a wrong answer with apparent confidence.

Reasoning models close this gap through training. Rather than relying on the prompt to elicit reasoning, the model is trained on examples that include detailed reasoning traces. During training, the model learns to produce effective reasoning chains, not just plausible-looking ones. It learns to break problems down, check intermediate results, and backtrack when a path looks wrong. This is achieved through reinforcement learning, where the model is rewarded for arriving at correct answers through valid reasoning steps.

How Reasoning Models Work

The mechanics of a reasoning model build on the same Transformer architecture as standard LLMs. The difference is in training and inference.

Pascal Academy

Pascal Academy's advanced LLM course covers reasoning model architectures, training methods, and deployment patterns. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

Training with Reasoning Traces

Standard models are trained on text where each token follows the previous one. Reasoning models are additionally trained on data that includes step-by-step reasoning traces. These traces show the model how to decompose a problem, how to check intermediate results, and how to arrive at an answer through logical steps. The training data can come from human experts who write out their reasoning process, or from automated systems that generate and verify reasoning chains.

Reinforcement learning plays a central role. The model generates reasoning traces for problems with known correct answers. Traces that lead to correct answers are rewarded. Traces that lead to wrong answers are penalised. Over many iterations, the model learns which reasoning strategies work and which do not. This is similar to how RLHF teaches a model to be helpful, but the reward signal comes from correctness rather than human preference.

Inference with Extended Thinking

At inference time, the model is given a question and generates a chain of thought before producing its answer. This chain of thought can be hundreds or thousands of tokens long for complex problems. The model explores different approaches, checks its work, and sometimes backtracks. Only the final answer is presented to the user in some interfaces, while the reasoning trace is hidden or shown in a collapsible section.

The amount of reasoning the model does is related to the difficulty of the problem. For a simple factual question, the model may produce a short reasoning trace or skip extended reasoning entirely. For a complex math problem, it may produce thousands of tokens of reasoning before committing to an answer. This adaptive compute is a defining feature of reasoning models.

Go deeper

Reinforcement learning and the reasoning reward

Optional

The training process for reasoning models uses a reward signal based on answer correctness. For math problems, the reward is binary, the answer is either right or wrong. For coding problems, the reward comes from test cases that pass or fail. For other tasks, the reward can be more nuanced, based on partial credit or human evaluation of the reasoning quality. The model learns through many rounds of generation and feedback, gradually improving its reasoning strategies. This process is expensive because each training example requires the model to generate a full reasoning trace, which takes more compute than a single answer. The cost is justified by the improvement in reasoning quality, but it means reasoning models are more expensive to train than standard models.

Reasoning Models vs Standard Models

Reasoning models and standard models serve different purposes. Understanding when to use each is a practical decision that depends on the task, the latency budget, and the cost tolerance.

Standard Model	Reasoning Model
Generates the answer directly from the prompt.	Generates a chain of thought, then the answer.
Fixed compute per token regardless of difficulty.	Scales compute with problem difficulty.
Fast response, low latency per query.	Slower response, higher latency due to extended thinking.
Lower cost per query.	Higher cost per query due to additional tokens generated.
Good for text generation, summarisation, extraction, chat.	Good for math, logic, coding, and multi-step analysis.
Can hallucinate on reasoning tasks because it cannot check its work.	Catches some of its own errors through the reasoning process.

The choice between a reasoning model and a standard model is rarely either-or. Many systems use both. A standard model handles the initial query, determines what kind of question it is, and routes complex reasoning tasks to a reasoning model. This routing approach gives you the speed of a standard model for easy questions and the accuracy of a reasoning model for hard ones.

What Reasoning Models Are Good At

Mathematics and Logic

Mathematical problems require precise, multi-step reasoning. Each step depends on the previous one, and a single error propagates through the entire solution. Reasoning models excel here because they can check intermediate results and backtrack when a step produces an unexpected value. On competition mathematics benchmarks like AIME and the International Mathematical Olympiad, reasoning models score significantly higher than standard models.

Coding and Software Engineering

Writing code requires understanding requirements, planning an approach, implementing it, and debugging when things go wrong. Reasoning models can plan the structure of a solution before writing code, consider edge cases, and reason about why a particular implementation might fail. On coding benchmarks like competitive programming problems, reasoning models outperform standard models by a significant margin.

Scientific Analysis

Scientific questions often require synthesising information from multiple sources, applying domain knowledge, and drawing conclusions through logical inference. Reasoning models can work through these steps methodically, making their reasoning process visible and auditable.

Complex Planning

Tasks that require breaking a goal into sub-tasks, ordering them, and accounting for dependencies benefit from extended reasoning. A reasoning model can plan a multi-step process, identify potential failure points, and adjust its plan before execution. This is useful for agent systems, covered in the AI agents guide, where the model needs to decide what actions to take and in what order.

What Reasoning Models Struggle With

Reasoning models are not universally better. There are tasks where the extended reasoning provides no benefit and only adds cost and latency.

Factual Recall

If the question is about a fact that the model either knows or does not know, reasoning does not help. The capital of France is Paris regardless of how many reasoning steps the model takes. Extended thinking on factual questions wastes tokens without improving accuracy. In some cases, it can even hurt, because the model second-guesses a correct answer it would have given immediately.

Creative Writing

Creative tasks like writing a poem, drafting a marketing email, or composing a story do not benefit from logical reasoning. These tasks require creativity, tone, and style, which are different capabilities from

logical problem-solving. A reasoning model may produce overly analytical or dry creative output because its training biases it toward structured thinking.

Simple Tasks

For straightforward tasks like summarising a document, classifying text, or extracting entities, the extra reasoning tokens are pure overhead. The model could answer just as well without them, and the user waits longer for no benefit. Using a reasoning model for every query is like hiring a research scientist to answer the phone. Sometimes you just need a quick answer.

Go deeper

Overthinking and the diminishing returns of reasoning

Optional

Research has shown that reasoning models can overthink problems. On tasks where the model already knows the answer, extended reasoning can introduce errors. The model starts questioning its initial intuition, explores alternative paths, and sometimes arrives at a worse answer than it would have produced immediately. This is the reasoning equivalent of a student who knows the answer but talks themselves out of it on a multiple-choice test. The optimal amount of reasoning depends on the difficulty of the problem, and finding that optimum is an active area of research. Some systems use a classifier to predict question difficulty and route accordingly, using a reasoning model only when the question is hard enough to warrant the extra compute.

Cost and Latency Trade-offs

The primary cost of reasoning models is the additional tokens they generate. A reasoning trace can be 10 to 100 times longer than the final answer. If the answer is 50 tokens and the reasoning trace is 5,000 tokens, the model is generating and you are paying for 100 times more output than you see.

On latency, the model generates tokens sequentially. A 5,000-token reasoning trace takes 50 to 100 seconds to generate at typical speeds, depending on the model and infrastructure. For interactive applications, this delay is significant. A user who asks a question and waits two minutes for an answer may lose patience, even if the answer is better than what a standard model would produce in two seconds.

API providers typically price reasoning tokens separately from output tokens, and some allow you to set a reasoning budget, a cap on how many tokens the model can spend thinking. This lets you control cost, but setting the budget too low can prevent the model from finishing its reasoning, leading to incomplete or wrong answers.

Pascal Academy

Pascal Academy's production AI course covers model selection, cost optimisation, and routing strategies for mixed reasoning and standard model deployments. Reach out to learn about team training options.

Ask about team training

Factor	Standard Model	Reasoning Model
Tokens generated	Answer only, typically 100 to 500 tokens.	Answer plus reasoning trace, often 1,000 to 10,000 tokens.
Latency	1 to 5 seconds for typical queries.	10 seconds to several minutes depending on reasoning length.
Cost per query	Low, proportional to input plus output tokens.	High, proportional to input plus reasoning plus output tokens.
Best use case	Chat, summarisation, extraction, simple Q and A.	Math, coding, multi-step analysis, complex planning.

When to Use a Reasoning Model

The decision to use a reasoning model depends on the task, the accuracy requirements, and the tolerance for cost and latency.

Situation	Recommendation
Math, logic, or scientific problem requiring multi-step reasoning	Use a reasoning model. The extended thinking significantly improves accuracy on these tasks.
Coding challenge with complex logic or algorithm design	Use a reasoning model. The model can plan the approach and reason about edge cases before writing code.
Document summarisation or text extraction	Use a standard model. No multi-step reasoning needed, and the reasoning model adds unnecessary cost and latency.
Chatbot for customer service with quick response requirements	Use a standard model for speed. Route to a reasoning model only for complex queries that the standard model cannot handle.
Research question requiring synthesis across multiple sources	Consider a reasoning model combined with RAG. The model can reason over retrieved documents and draw conclusions through logical steps.
Creative writing, marketing copy, or content generation	Use a standard model. Reasoning models tend toward analytical output that can stifle creativity.
Mixed workload with varying question difficulty	Use a routing approach. A classifier or the standard model itself determines whether to escalate to a reasoning model based on question complexity.

Reasoning Models and Hallucination

The previous guide covered hallucination, where models produce confident but factually wrong output. Reasoning models offer a partial improvement here, but the relationship is nuanced.

On reasoning errors, where the model makes a logical mistake or calculation error, the extended chain of thought helps. The model can catch errors in its own reasoning, check intermediate results, and correct course. This reduces errors on math, logic, and coding tasks.

On factual hallucination, where the model states something untrue, reasoning provides less help. If the model does not know a fact, thinking longer does not make the fact appear. The model may reason about the fact, drawing inferences based on adjacent knowledge, but those inferences can be wrong. Extended reasoning can even amplify factual hallucination, because the model generates more text and has more opportunities to introduce unsupported claims.

The practical takeaway is that reasoning models are not a substitute for grounding. For factual accuracy, RAG and knowledge graphs remain the primary tools. Reasoning models complement these by improving the model's ability to reason over the retrieved information, but they cannot conjure facts that are not in their training data or in the provided context.

The Future of Reasoning Models

Reasoning models represent a direction in AI development where performance improves by spending more compute at inference time rather than only at training time. This is a meaningful change because it decouples capability from model size. A smaller model that reasons longer can outperform a larger model that answers immediately.

Current reasoning models still have limitations. Their reasoning traces can be verbose and hard to parse. They sometimes reason in circles or fail to converge on an answer within their token budget. The quality of reasoning depends heavily on training data, and models can learn reasoning patterns that look correct but contain subtle logical errors.

Research is active on several fronts. Techniques for more efficient reasoning, where the model produces shorter and more focused chains of thought, could reduce cost and latency without sacrificing accuracy. Approaches that combine reasoning with tool use, where the model can call external tools like calculators, code interpreters, or databases during its reasoning process, extend what the model can

accomplish. And the integration of reasoning models with agentic systems, where the model plans and executes multi-step workflows, is an area of rapid development.

Summary

A reasoning model is a language model trained to produce an extended chain of thought before generating its answer. The model spends additional tokens, called test-time compute, thinking through the problem. This improves performance on tasks requiring multi-step reasoning like math, coding, and logical analysis. The trade-off is cost and latency, because the model generates many more tokens than a standard model would for the same question. Reasoning models are trained using reinforcement learning with rewards based on answer correctness, teaching the model to produce effective reasoning strategies. They are not universally better than standard models. For factual recall, creative writing, and simple tasks, the extra reasoning provides no benefit and only adds overhead. Many production systems route between standard and reasoning models based on question difficulty. Reasoning models help with reasoning errors but do not solve factual hallucination, because thinking longer cannot produce facts that are not in the model's training data or context. For factual accuracy, grounding through RAG and knowledge graphs remains essential.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, context windows, knowledge graphs, hallucinations, reasoning models, embeddings, and MCP. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is a reasoning model in AI?

A reasoning model is a type of AI that breaks down complex problems into steps before producing an answer. Instead of generating a response immediately, it plans an approach, works through intermediate steps, and arrives at a conclusion. This makes it better at tasks that require multi-step logic.

How are reasoning models different from standard LLMs?

Standard LLMs generate a response token by token in a single pass. Reasoning models spend additional compute thinking through the problem before answering. They produce intermediate reasoning that is visible to the user, which makes the output easier to verify and debug. The trade-off is that reasoning models are slower and more expensive per query.

How does Pascal use reasoning models for investment research?

Pascal's research agents use structured reasoning to break down complex investment questions. When you ask which companies in a universe are expanding margins while maintaining growth, the agent plans the analysis, retrieves relevant financials, evaluates each company, and synthesizes the results with citations. The reasoning is traceable, so you can see how the agent arrived at its conclusion.

What are the limitations of reasoning models?

Reasoning models are slower and more expensive than standard models. They can still make errors in their reasoning chain, and the visible reasoning steps do not guarantee correctness. They are best suited for complex analytical tasks where the extra time produces a meaningfully better result, not for simple lookups or drafting tasks.